EasyMeals

By: Dean Leon, Ayowade Owojori, Dylan Peniuk, Noam Yaffe

Team information

Roles + Reasoning

- Dean Leon - Full-stack developer → Has experience on both sides of the development stack and can help with tasks on all ends
- Ayowade Owojori - Backend lead → Has participated in a lot of projects surrounding backend development, and feels comfortable with the tools that we're using
- Dylan Peniuk - Project leader → Came up with the idea, and has the most concrete view on what the project should do, what it should look like, etc.
- Noam Yaffe - Frontend lead → Most experience with UI/frontend design and implementation

Member Responsibilities

Name	Role	Responsibilities
Dylan Peniuk	Project leader	-Ensure the project is fulfilling all of the functional requirements -Work on the look of the website, making sure it is easily traversable and accessible for anyone. -Work alongside Noam for the frontend development and implementation -Making sure the dashboard is well-made and readable, and the correct information about recipes and prices is present.
Noam Yaffe	Frontend lead	-General frontend oversight,

		implementation, and guidance. -Will specifically implement a working demo of our homepage, login/signup page, and questionnaire. -Will work alongside Dylan to implement the Dashboard UI and component logic.
Ayowade Owojori	Backend lead	**Meal Retrieval & Scoring** 1. Query meals by structured filters 2. Compute Bayesian recommendation score 3. Compute user-specific diversity weight 4. Compute final selection score 5. Sample top-K meals probabilistically **Plan Assembly** 1. Assemble weekly/monthly meal schedule 2. Aggregate grocery list 3. Recalculate total cost 4. Enforce budget constraint 5. Validate macro goals **Personalization Layer** 1. Store per-user recent meal window 2. Implement rolling window logic 3. Apply diversity decay dynamically
Dean Leon	Full-stack developer	**AI & Prompt Layer**

		1. Design strict JSON schema for Gemini responses 2. Build prompt template 3. Implement Gemini connection in FastAPI 4. Validate and sanitize AI output **Database Management** 1. Insert new meals with deduplication check 2. Normalize ingredients 3. Update rating stats 4. Update recommendationScore on new rating 5. Track user meal history with timestamps **Reliability & Safety** 1. Rate limit Gemini calls 2. Handle malformed AI responses 3. Add logging 4. Add retry logic 5. Fallback behavior if no meals match constraints

GitHub Repository

- https://github.com/yaffenator/EasyMeals

Communication Channels

- Microsoft Teams

Product description

Abstract
Meal prepping is hard. For any individual receiving food benefits or other financial aid, the idea of properly allocating a portion of their monthly income to groceries as effectively and efficiently as possible may seem like a daunting task. Introducing: EasyMeals – Automate your meal-prepping goals with a personalized AI nutrition expert. This website will allow you to enter the amount of money you allocate to food and groceries each month and select your physical goals (such as losing weight or building muscle) and their extent. Using this information, EasyMeals will create a month-long meal-prepping plan, showing you what groceries to buy, how much of each grocery item you will need, and some fun meals that you can prepare to satisfy your hunger.

Goal
The goal of this project is to make a website that allows people to skip the trouble of budgeting for food and the stress of finding recipes to prepare, by having it all in one place. It will allow people to enter their monthly food budget and automatically give a list of groceries to buy and recipes of what to cook week by week. It will also ask for food allergies and restrictions to let people be stress-free about what they are eating. Overall, this project would reduce the stress of wondering what you are able to afford and what you are able to make, by laying it all out month by month.

Current practice
Currently, meal planning is dominated by subscription-based meal kits that, while convenient, operate at a significant financial premium. According to a CNET article, these meal kits cost an average of $9 to $11 per serving compared to the $5 to $7 per serving cost of traditional grocery shopping (CNET 2025). Because most meal kits and pre-made delivery services are classified by federal guidelines as prepared or subscription food services, they are ineligible for government assistance programs like EBT/SNAP, which strictly require funds to be spent on unprepared grocery staples (Campbell 2025).

Novelty
What makes our approach different is the introduction of an AI assistant that will webscrape products from the internet to create specialized meal plans that are applicable to user requests. This approach will factor in the cost of ingredients, which is absent from existing services, and the ability to cook the meals yourself, which is not available in services like “Factor_” that ship the entire meal.

Effects
While optimizing grocery expenses is a universal benefit, EasyMeals is purposefully designed to support vulnerable demographics, specifically lower-income households and individuals managing multiple jobs. EasyMeals addresses the unique barriers faced by these groups in the following ways:

- For Lower-Income Households: EasyMeals directly alleviates budget anxiety by scraping real-time pricing from EBT-approved grocery stores. The AI ensures that every generated meal plan strictly adheres to the user's predefined financial limits, eliminating the markups of standard meal kits

- For individuals working multiple jobs: When working multiple jobs, the primary barrier to home cooking is the lack of time. EasyMeals addresses this by automating the cognitive task of meal planning. Using EasyMeals, users receive instant, appropriate meal plans and consolidated grocery lists

---

Product Use Cases

1. (Noam Yaffe) The user will narrow down their health requirements & fitness goals through a variety of health-related inquiries.
    a. Actor: Registered user
    b. Triggers: Upon registering for the website, the user is immediately met with health/fitness-specific questions
    c. Preconditions:
        i. The user is logged into the system.
        ii. The user knows the state of their physical well-being & health requirements, whether that is their body weight, their weekly activity level, any allergies they have, their food preferences, etc.
    d. Postconditions:
        i. The user has properly responded to all health & fitness related questions.
        ii. The system will use this information to fit the generated meal plan to the user's health and fitness goals.
    e. Success Scenario:
        i. The user successfully answers all questions relating to their fitness & health goals
        ii. The system fine-tunes the generated meal-prepping plan based on the user's health & fitness inputs.
    f. Extensions &Variations:

i. Weight loss vs weight gain: depending on the user's goals, they could additionally list the amount of weight that they hope to gain or lose within the specified one-month timeframe.
ii. Activity level descriptions: if time allows, we can additionally ask the user to describe their weekly activity level through the activities they participate in, which would give our model further insight into their current physical activity and needs.

g. Exceptions:
i. Invalid inputs: The user enters information that the system cannot process
ii. Faulty API call: Each user receives a designated API request to our AI model, which generates a meal plan tailored to their needs. However, whether this works depends on the API key functioning properly alongside other factors.

2. <u>(Dylan Peniuk) The system identifies SNAP-eligible items within a generated grocery list and tracks the remaining monthly benefit balance to ensure the user never exceeds their allotted aid.</u>
a. Actor: Registered user
b. Triggers: The user generates an ingredient list
c. Preconditions:
i. The user has entered their monthly SNAP benefit in their profile
ii. A meal and grocery list has been created
d. Postconditions:
i. The grocery list is visually partitioned into "EBT Eligible" and "Cash Required" sections.
ii. The system confirms the total eligible amount is less than the user's remaining SNAP balance.
iii. The user is notified of any "out-of-pocket" costs
e. Success Scenario:
i. Activate Filter: The user enables "EBT Optimization" in their plan settings.
ii. Item Validation: The system cross-references the generated grocery list against the SNAP-eligible food list
iii. Balance Calculation: The system subtracts the estimated cost of eligible items from the user's current "Benefit Balance."
iv. Item Tagging: The system generates the final list, marking eligible items with a specific icon
v. Out-of-pocket summary: The system provides a secondary total for items that require cash or debit
vi. Confirmation: The user reviews the breakdown and saves the list

f. Extensions &Variations:
    i. Combined Payment View: For users who use both SNAP and cash, the system provides a "Split-Payment" preview so they know exactly how much will be charged to each card at the register.
g. Exceptions:
    i. Benefit depletion: The generated plan exceeds the monthly SNAP balance
    ii. Database sync error: The system cannot verify the SNAP status of a new or niche product

3. (Dean Leon) The user will receive an itemized grocery list and curated recipes tailored specifically to their financial constraints and physical objectives.
    a. Actor: Registered user
    b. Triggers: The user generating the plan (i.e., clicking a button)
    c. Preconditions:
        i. User is logged in
        ii. User has completed their health profile
        iii. Grocery price data is available
    d. Postconditions:
        i. The user receives a downloadable/viewable grocery list.
        ii. The user receives a curated meal plan where the total cost is <= User Budget.
        iii. The total nutritional output matches the user's physical objectives.
    e. Success Scenario (Main Success Path):
        i. Input Parameters: User enters their maximum budget and goal.
        ii. System Calculation: The system filters recipes that fit the nutritional macro-profile.
        iii. Cost Optimization: The system cross-references ingredients with current market prices to find the most cost-effective combination.
        iv. Review: The system presents the proposed meal plan and total estimated cost to the user.
        v. Finalization: The user accepts the plan.
        vi. Delivery: The system generates an itemized grocery list organized by store aisle and provides step-by-step cooking instructions for recipes.
    f. Extensions & Variations:
        i. Dietary Restrictions: The user can toggle filters for vegan, gluten-free, or keto-friendly options within the same budget.
        ii. Inventory Integration: The user can "check off" items they already have at home (e.g., salt, oil, rice) to further reduce the total cost.
    g. Exceptions (Failure Scenarios):

i. Insufficient Budget: The user's budget is too low to meet their caloric/nutritional needs
ii. API Timeout: Real-time grocery pricing data is unavailable.
iii. Conflicting health goals

4. (Ayowade Owojori) The website will prioritize grocery stores within the user's physical proximity, and use them as a baseline for item pricing & meal-prepping.
    a. Actor: Registered user
    b. Triggers: The user inputs (or updates) their zip code location
    c. Preconditions:
        i. The user has entered (or updated) their location via zip code
        ii. The user completed the fitness and health goals questionnaire
    d. Postconditions:
        i. The system finds grocery stores in proximity to the user's inputted zip code, and uses them to estimate the costs of certain food items
    e. Success Scenario:
        i. The system creates a meal-prepping plan that also accounts for the user's proximity, alongside their budget constraints and fitness goals
    f. Extensions and Variations
        i. The system can use stores in proximity to the user to compare different food items' prices, therefore generating the most cost-effective, location-based meal-prepping plan for the user
    g. Exceptions
        i. Location denied: The user refuses to share location or zip code
        ii. No supported retailers: The user is in an area where food prices can be determined online
        iii. Stale data: The local price feed hasn't been updated in over 24 hours.
        iv. The user is outside the supported region of the program

---

Product Requirements & Technical Approach

Non-functional Requirements

- **Scalability**: The system must be able to handle up to 1,000 meal-prep generation requests per day, with the ability to support up to 50 concurrent users without performance degradation.
    - **Justification**: 1,000 daily requests allows for a healthy MVP user base. The 50 concurrent user metric is crucial because meal planning is highly seasonal

throughout the week. Traffic will heavily spike on weekend afternoons and evenings before the workweek begins, rather than being evenly distributed throughout the day.

- **Performance**: The system must generate and display the customized meal plan and consolidated grocery list within 15 seconds of the user submitting their parameters.
    - **Justification**: Because EasyMeals relies on live web-scraping of local grocery store prices and AI processing, instant generation isn't feasible. A 15-second benchmark balances the heavy backend processing required with user attention spans, likely requiring the system to cache frequently searched grocery items to meet this speed.

- **Security**: The system must encrypt all sensitive user profile data, specifically budgetary limits, dietary restrictions, and health goals, at rest using AES-256 encryption and in transit using TLS 1.2 or higher.
    - **Justification**: To generate truly customized meal plans without forcing the user to re-enter their complex preferences, financial limits, and health goals every single time they log in, EasyMeals must persistently store this profile data. Because storing this information inherently creates a record of an individual's financial status and personal health habits, the system requires strict, measurable cryptographic protocols.

External Requirements

- Robust Input and AI Output Validation: The backend must validate all user-provided inputs (budget, allergies, dietary goals, calorie targets) and enforce strict schema validation on Gemini-generated responses before storing or returning meal plans. The system must gracefully handle malformed AI responses, API timeouts, and Firebase read/write failures.

- Publicly Accessible Web Deployment: EasyMeals must be deployed as a publicly accessible web application, consisting of a hosted Next.js frontend and FastAPI backend integrated with Firebase Authentication and Firestore for persistent storage.

- Reproducible Build and Configuration**:** The entire system must be buildable from source by external developers. Documentation must include setup instructions for Firebase configuration, Gemini API credentials, environment variables, database schema initialization, and backend deployment.

- External Service Dependency Management: The system relies on Google Gemini for meal plan generation and Firebase Firestore for data persistence and querying. The backend must handle API latency, rate limits, and service interruptions without compromising data integrity.

- Resource-Aligned Feature Scope: The implemented functionality must align with the team's size and timeline, focusing on AI-generated meal plans, budget-aware grocery aggregation, Bayesian recommendation scoring, and per-user diversity personalization as the core deliverables.

Technical approach

Our approach is to build EasyMeals as a full-stack web application with a clear separation between frontend presentation, backend logic, and external AI services. The frontend will provide an interactive interface where users input budgets, dietary restrictions, and physical goals. The backend will handle validation, business logic, database interaction, recommendation scoring, and communication with the Gemini API for meal generation and reasoning.

For our software architecture, we chose a layered architecture model. Implementing a layered architecture provides critical advantages in maintainability, development velocity, and system security by isolating the application's core components. Because each layer operates independently, modifying one section does not require a cascading rewrite of the entire system; for instance, if the decision is made to swap the underlying AI model from Gemini to Claude, only the specific backend logic needs updating, while the frontend interface and database remain completely untouched. This strict separation of concerns also enables parallel development, allowing teams to build the user interface and backend services simultaneously without conflict, provided the API interfaces between them are clearly defined. Furthermore, this architectural isolation drastically streamlines the testing process by allowing for highly targeted unit tests. Specific modules, such as the budget calculator, can be rigorously tested in isolation, entirely bypassing the need to launch the full website or connect to the internet for the web-scraping data layer. Finally, a layered approach inherently fortifies the application's security posture. Because users only interact with the presentation layer, all their requests must pass through the API—which acts as a secure gatekeeper to validate and sanitize the input—ensuring that no direct, malicious, or malformed data can ever reach the backend database.

Development will proceed in staged phases over the project timeline. First, we will establish the database schema and implement core backend endpoints for meal storage, scoring, and retrieval. Next, we will integrate Gemini for structured meal plan generation and implement input validation and error handling to ensure robustness. After core functionality is stable, we will implement personalization features such as diversity tracking and recommendation scoring.

Finally, we will deploy the application to a publicly accessible server, conduct integration testing across the frontend, backend, and AI components, and refine performance and reliability.

The system will be publicly hosted, with all components buildable from source using documented setup instructions for Firebase configuration, API credentials, and environment variables. Authentication will be implemented using Firebase Authentication to ensure that user preferences, ratings, and meal history data are securely associated with individual accounts.

Product Risks

While the content for each risk listed below has been revised since we submitted our Requirements document (to ensure further detail and specificity), the risks themselves have largely stayed the same and proved to be crucial to our project's security and success.

Risk #1: Budget & Pricing Inaccuracy

- Likelihood: **HIGH**. Grocery prices fluctuate frequently based on inflation, season, and location – these can all impact the accuracy of the generated meal plan and its adherence to the user's food benefit limitations.
- Impact: **HIGH**. If a user on a strict budget (like EBT/SNAP) is led to the store and cannot afford the ingredients, then our website has essentially failed its primary goal.
- Evidence: Existing research into food benefits (EBT/SNAP) shows that even small price discrepancies can lead to benefit depletion before the month ends.
- Steps to Reduce: While we cannot control the prices of specific grocery items, we can remain aware of them. The USDA's Economic Research Service releases an Excel document showcasing the Changes in Consumer Price Indexes (last updated January 23, 2026) each month. We can therefore use this resource and manually feed it into our model to update its grocery pricing conventions & calculations, ensuring up-to-date meal pricing outputs.
- Detection & Mitigation: Every time a meal plan is generated, its total expected price will automatically be compared with the user's inputted monthly benefits. If the predicted price of the meal plan exceeds the user's food benefits, the system will immediately regenerate a new meal plan for the user.

Risk #2: Failure to Adhere to Dietary Restrictions

- Likelihood: **MEDIUM**. LLMs like Gemini can occasionally experience "hallucinations" or overlook constraints in complex prompts.

- Impact: **HIGH**. Including an allergen (e.g., peanuts or gluten) can lead to serious health issues and liability.
- Evidence: During initial testing of Gemini API prompts, we identified that without explicit "hard constraints," the model might prioritize a recipe's "quality score" over a niche restriction.
- Steps to Reduce: We have made the health questionnaire MANDATORY during registration. We are fine-tuning our prompts to treat allergies as "Hard Constraints" that override all other meal-scoring algorithms
- Detection & Mitigation: We are using adversarial testing with synthetic user profiles (e.g., a user with 5+ severe allergies) to ensure 100% of generated plans satisfy the constraints before they are displayed. If a restricted item is detected in a plan, the system will automatically trigger a meal plan regeneration to replace the specific recipe without requiring user intervention

Risk #3: Data Exposure / Leaking of Sensitive Health Data

- Likelihood: **LOW**. By using managed cloud services, the probability of a breach is minimized compared to self-hosted solutions.
- Impact: **HIGH**. Our website will collect sensitive metrics like body weight and fitness habits, which require strict privacy.
- Evidence: Reviewing Firebase's security documentation confirms that it provides robust, industry-standard encryption for both data at rest and in transit.
- Steps to Reduce: We are utilizing Firebase Authentication and Firestore security rules to ensure that users can only access their own data. No sensitive health data will be sent to the Gemini API unless it is strictly necessary for the prompt logic.
- Detection & Mitigation: We will perform Gray Box testing and database validation to ensure there are no unauthorized access points between user collections. In the event of a security flaw, we will immediately revoke API keys and use Firebase's administrative console to lock down data access while patches are applied.

Risk #4: API Failure or Service Timeout

- Likelihood: **MEDIUM**. The project relies heavily on the Gemini API for core logic and external grocery data feeds, which can be subject to rate limits or temporary outages.
- Impact: **HIGH**. A faulty API call or timeout prevents the generation of the month-long meal plan, rendering the Personalized Dashboard non-functional for the user.
- Evidence: We have already noted "Faulty API call" and "API Timeout" as critical exceptions in our project's use cases.

- Steps to Reduce: We are implementing a Gen-AI SDK with built-in error handling. We are also caching common meal plan templates to provide "fallback" options if a real-time request fails.
- Detection & Mitigation: We will use Pytest for backend function tests to monitor API response times and status codes during the integration phase. If an API call fails, the system will notify the user of the delay and offer a high-quality "Standard Plan" from the database until service is restored

Risk #5: Overwriting Another Person's Work/Features During Development

- Likelihood: **HIGH**. Given that we have four members working on different layers (Frontend, Backend, and Full-stack) simultaneously, the probability of two people touching the same configuration files, API routes, or shared CSS files is very high.
- Impact: **MEDIUM**. While this risk won't necessarily break the entire project permanently, it can cause significant delays, lose hours of individual progress, and introduce regressions where previously working features suddenly stop functioning.
- Evidence: In software engineering, parallel tracks without strict version control protocols almost always lead to lost updates or dirty writes in the codebase. While this is evidence in all workplaces, we need to make sure we mitigate its possibility in our own codebase.
- Steps to Reduce: We will ensure separate branches are made for new features and then merged into our main branch to prevent us from losing those features.
- Detection & Mitigation: We can use GitHub Actions to run automated tests on every PR. If a new piece of code breaks an existing feature (Regression), the test suite will fail and alert the team immediately. Alongside this, no code will be merged into the main branch without a Peer Review from another team member to ensure that the incoming changes don't accidentally delete or overwrite existing code/logic.

Software architecture

Chosen software architecture: **Layered architecture pattern**

Identify and describe the major software **components** and their functionality at a conceptual level.

- User-Facing Frontend: Built with React and Next.js, this specific component will handle the user interface, visualizing all user interactions within our website.
- Application Server: A Python-based backend that acts as the brain of the program. Takes the user-inputted data regarding their health & fitness, constructs optimized prompts to

send to our Gemini API, and receives the API response to be refactored and displayed to the user.
- Gemini API: Receives the data from the business logic and returns a formatted JSON object containing the month-long meal plan and grocery list.
- Database Backend: A series of collections stored in a Firestore database, containing instances and attributes that correspond to different user profiles, meal plans, and saved grocery lists.

Specify the **interfaces** between components.

- Frontend ↔ Backend: The frontend sends JSON payloads of user preferences, and the backend returns structured meal data.
- Backend ↔ Gemini API: Utilizes Google's Gen-ai SDK to send prompts and receive structured text and JSON responses.
- Backend ↔ Database: Uses Firebase to perform CRUD operations on user data.

Describe in detail what **data** your system stores, and how. If it uses a database, give the high-level database schema. If not, describe how you are storing the data and its organization.

- To store our data, we will connect our web application to a Firestore database (Firebase). This development platform provides a NoSQL, Schema-less database for us to store user information, meal plans, and other important data. That said, since the database does not directly use a Schema to operate, we will instead provide a high-level description of which collections will store what attributes, and the specific data to be stored inside of them. This will be updated with more specific details once the backend is set up, and when we've fully settled on what user & meal data we will be storing in our database.

If there are particular **assumptions** underpinning your chosen architecture, identify and describe them.

- Assumes a persistent internet connection for Firebase and Gemini uptime.
- Assumes grocery prices that are scraped are updated at least every 24 hours.
- Assumes user is located in the US.
- Assumes that the user has an idea of their current health status and their future meal-planning & fitness goals.
- Assumes that the user knows the specific amount of monthly benefits that they receive for personal food security.

For each of two decisions pertaining to your software architecture, identify and briefly describe an **alternative**. For each of the two alternatives, discuss its pros and cons compared to your choice.

- Alternative 1: Microservices Architecture
    - Splits the AI logic, user management, and grocery scraping into separate deployable services.
    - Pros: Highly scalable, if one aspect crashes or goes down, the rest of the application will stay up
    - Cons: Overly complex for a team of 4. High overhead in managing inter-service communication and deployment.

- Alternative 2: Client-server architecture
    - Runs all logic, including AI calls and data processing from the React frontend, using Firebase as a direct backend service.
    - Pros: Faster, no need to maintain a Python backend
    - Cons: Significant security risks in exposing API keys, poor performance on mobile devices, and limited web scraping capabilities.

Software design

What packages, classes, or other units of abstraction form these components?

- Frontend (Next.js/React):
    - Components/
    - Context/

- Backend (Python/Flask or FastAPI):
    - Scraper/
    - AI_Engine/

- Database (Firebase):
    - Firestore
    - Firebase Auth

What are the responsibilities of each of those parts of a component?

- Components/ will hold reusable UI elements

- Context/ will hold global user states
- Scraper/ will hold modules dedicated to finding princes and information from local retailers
- AI_Engine/ will hold the logic for crafting and fetching the prompts from the Gemini API
- Firebase will be realtime storage
- Firebase Auth will hold OAuth and email/password tokens

Coding guideline:

Coding guidelines for the programming languages, frameworks, and platforms we will use:

- TypeScript: Follow coding conventions from TypeScript's official documentation: https://www.typescriptlang.org/docs/

- TailwindCSS: Follow documentation to create a clean, professional-looking UI: https://tailwindcss.com/

- Python: Follow PEP 8 coding standards: https://peps.python.org/pep-0008/

- React: Follow the library's strengths & utilization conventions as provided in the documentation: https://react.dev/

- Firebase: The development platform has a lot of documentation surrounding its database and provides authentication systems: https://firebase.google.com/

Product Schedule

Project Milestones

Milestone	Tasks	Prerequisites
User Interface	-Build a cohesive, professional landing page. -Landing page includes the purpose of the project, how the project works, etc.	-Set up codebase (DONE) -Choose frontend tools, frameworks, and libraries (DONE)

	-Create a section for users to sign up or log into their EasyMeals account	
Authentication System	-Create a working, user/password-based authentication system for users to create an account or log into a preexisting one -If time permits, add an option to implement Google OAuth.	-An authentication system has been chosen for the website's auth to be implemented with. -UI has been created for the authentication page of the website.
Health & Fitness Questionnaire	-Create a UI for users to enter information about their health & fitness status and goals. -Store information in the database.	-Choose a database to store user-specific information (DONE) -Connect the working database to the project website.
User Dashboard	-Generate a monthly, week-by-week meal-prepping plan specific to the user's answers in the questionnaire. -Display this meal-prepping plan through a coherent, professional-looking UI.	-UI & Health Questionnaire. -Working database connected to the project website.
Gemini API	-Generate API Key -Choose model -Share API key with members and set up environment variables on all development machines	-Python environment configured.
Core AI Logic	Develop backend prompt engineering to convert user data into a formatted JSON object	-Gemini API & Questionnaire data.
Grocery and Recipe Engine	Implement grocery list generation and recipe formatting; integrate basic web-scraping for pricing.	-Core AI Logic.

Basic Project Timeline (From Week 6→ Submission)

Week 6: Midterm presentation

- **All members:** Deliver the midterm project presentation.
    - Dylan: Intro and idea
    - Noam: UI, Risks, and team introduction
    - Ayowade: Algorithms
    - Dean: Software architecture and testing strategy

Week 7: Core Development & Integration

- **Wade and Dean:** Set up the codebase and tech stack (React, Next.js, TypeScript). Build the backend for user input requirements, implement the Gemini API, and begin end-to-end (E2E) integration to ensure data flows smoothly from user input to the final displayed meal plan.

- **Noam and Dylan:** Outline, implement, and finalize the UI components and web pages, ensuring the complete design implementation.

Week 8: & Initial Testing

- **All members:** Begin full system testing—including backend function tests and code merging—and gather external feedback to guide the remaining development.

Week 9: Final Testing & MVP Polish

- **All members:** Finalize system testing to ensure the base product is fully functional and stable. Once the core application is running smoothly, move on to implementing the listed stretch goals.

Week 10: Project Wrap-Up

- **All members:** Finalize the complete software package and set up the final project presentation.

Testing, Continuous Integration, and Documentation

Overall Test Plan & Bugs Report

We want to do **E2E** tests for the following using either **Playwright** or **Vercel's agent-browser**:

- Authentication
    - Testing that the user login, signup, and signout work without errors
    - We want to test this because if this breaks, users lose control over their accounts, and that would be a crucial bug.

- Meal-plan generation
    - Testing that the new changes that are made don't break the AI generation of the meal plans.
    - This is the main attraction of our website; if this is not working, our website is of no use, therefore, we need to always ensure our generation works.

Github Issues Link: https://github.com/yaffenator/EasyMeals/issues

- We will use GitHub issues to keep track of progress towards specific milestones and features being implemented, and to report any bugs that individual developers encounter along the way.

Continuous Integration

Our CI service and how our project repository is linked to it

- Our project uses **GitHub Actions** as our Continuous Integration service. GitHub Actions is a free automation tool that is built directly into GitHub, allowing us to automate our software development workflows in the same place we store our code.
- Our CI process is defined in two YAML files located at .github/workflows/frontend-tests.yml and .github/workflows/backendtests.yml. These files contain the instructions that GitHub Actions follows every time there is a push to the main branch or a pull request is opened.

Why we chose GitHub Actions

- Since our codebase resides on GitHub,using GitHub Actions for CI is a natural fit. It eliminates the need for third-party services and the associated setup and management overhead.
- For open-source projects and smaller teams, GitHub Actions offers a generous free tier that includes a significant number of build minutes per month. This makes it a cost-effective solution for our project

Pros/cons matrix for at least three CI services that you considered.

CI Services	Pros	Cons
Travis	It's simple to set up and compatible with python which we are using for our project. It's good at testing multiple versions of a language at one time. Lots of documentation and online support because of how long it's been around for and how many people have used it.	The free tier for this is pretty restrictive and doesn't allow for as much as GitHub Actions We are only allowed 10,000 credits, which doesn't allow for much expansion More into expansion, we are only allowed what is in the .travis.yml environment, and adding more tools can get messy
CircleCI	It can split large tasks into smaller parts and do them simultaneously, making tests for larger items much quicker It can cache parts of the environment to make loading certain dictionaries much faster and reduce loading times	It's very complex to learn how to use it well, and it doesn't fit the timeframe that we have for this project. It uses credits to test our software, and we have a lot of libraries that may take a lot of them, which could get expensive.
MS Azure pipelines	It provides very good traceability, where you can trace lines of code back to the build and test run it was created in. We could add our Firebase and Gemini keys behind a firewall for more security. It has a drag-and-drop UI and YAML, which makes it more accessible	It can be hard to see what is happening if different people are using the UI in different ways, making it difficult for quick testing There aren't as many new features so it seems like it would be fine to use as an existing CI but not something you should pick up.

Which tests will be executed in a CI build

- For backend: any .py files under the tests directory will be run
- For frontend: any .tsx testing files marked under a __tests__ directory will be tested

Which development actions trigger a CI build

- Whenever there's a pull request or a push to main, GitHub Actions automatically run the test suite and report the results

Frontend Testing Automation

Your test-automation infrastructure (e.g., JUnit, Mocha, Pytest, etc).

- For our test-automation infrastructure, we've chosen to implement Jest alongside the React Testing Library. Jest is a comprehensive JavaScript testing framework that is responsible for running the tests we configure in our test.js file and giving us feedback in the terminal on which tests passed or failed. On the other hand, the React Testing Library is a lightweight utility focused on what the user sees rather than how the code works internally, providing us with tools to virtually mount our React components into a DOM-like environment and other utilities to simulate user events and behaviors like clicks, typing, and hovering.

A brief justification for why you chose that test-automation infrastructure.

- We chose to use Jest and the React Testing Library because they allow us to write tests that are resilient to refactoring and properly mimic regular user behavior. For example, in any regular frontend test, Jest will set up the environment while the React Testing Library renders our frontend component and interacts with it as a regular user would. The React Testing Library then finds the resulting output from these frontend interactions, and Jest finally checks that output against our preset result expectations and reports whether or not the test succeeded.

How to add a new test to the code base.

1. Navigate to test directory: Whichever component or page you wish to test, navigate to its directory and locate the **__tests__** folder. If it does not exist, create one.

2. Create a test file: Once inside this directory, create a new file called [component_name].test.tsx (switch [component_name] with the name of the component).
3. Import necessary libraries: At the beginning of your test file, you need to import the necessary libraries. This will typically include React, render, screen, and fireEvent from @testing-library/react, and @testing-library/jest-dom for additional matchers.
4. Mock dependencies: If your component has dependencies, such as next/navigation for routing, you'll need to mock them. You can use jest.mock() to replace the actual implementation with a mock implementation.
5. Write the test suite: Use the describe function to group related tests for a component. The first argument is a string that describes the component you are testing, and the second argument is a callback function that contains the individual tests.
6. Write individual tests: Use the it or test function to define individual tests. The first argument is a string that describes what the test is checking, and the second argument is a callback function that contains the test logic.
7. Render the component: Use the render function from @testing-library/react to render your component in a virtual DOM.
8. Find elements: Use the screen object from @testing-library/react to find elements in the rendered component. You can use queries like getByText, getByRole, getByLabelText, etc.
9. Simulate events: If you need to test user interactions, you can use the fireEvent object from @testing-library/react to simulate events like clicks, changes, etc.
10. Make assertions: Use the expect function from Jest along with matchers from @testing-library/jest-dom to make assertions about the state of your component. For example, you can check if an element is in the document, if it has a certain text content, etc.
11. Manual testing: If you need to run the tests manually, you can navigate to the /frontend directory and run "**npm test**".

Backend Testing Automation

Your test-automation infrastructure (e.g., JUnit, Mocha, Pytest, etc).

- Pytest

A brief justification for why you chose that test-automation infrastructure.

- We chose Pytest because our backend is written in Python, making it the natural fit. Pytest has minimal repetitive code compared to alternatives like unittest because test functions are just plain functions prefixed with 'test_', no classes or setup inheritance required. It also has built-in support for 'unittest.mock', which we use extensively to

mock Firestore, and the pytest-cov plugin gives us code coverage reporting out of the box.

How to add a new test to the code base.

1. Navigate to backend/tests/ and open the test file corresponding to the service you want to test (e.g. test_ingredient_service.py for ingredient_service.py).
2. If testing a new service file that doesn't have a test file yet, create test_{service_name}.py in the ‘backend/tests/’ folder, and pytest will discover it automatically.
3. Write a new function prefixed with test_ that follows this pattern:
    a. Create a MagicMock() configured to return whatever Firestore would return for that scenario
    b. Patch the real database with your mock using with patch("db.your_service.db", mock_db)
    c. Call the function you're testing
    d. Assert the result is what you expect
4. Run pytest from the repo root to verify your test passes locally before pushing. The CI will run automatically on push and PR

Documentation plan

- We hope that the user can navigate through the system without any documentation or guide on how to use the website. As for developer guides, we will include a docs folder with instructional markdown files. For example, we’ll have a quick start markdown file that will have steps for starting up the website locally. Additionally, for any important setup steps for the Firebase database or the Gemini API setup, we will create markdown files with instructions.

Mid-Term Project Presentation Report

What problem(s) are you solving?

- Meal prepping is hard. Specifically for an individual receiving food benefits such as SNAP or other financial aid, meal prepping isn't just a health trend—it’s a financial necessity. With these circumstances in mind, the task of curating a meal plan that contributes to one’s health & fitness goals while abiding by financial restrictions can become a pretty daunting task. That being said, the **hardship** aspect of this ultimately necessary task is the problem that we want to mitigate.

How will your solution solve the problem(s)?

- Our website will allow people to easily find recipes that fit their budget, goals, and dietary requirements through personalized questions and AI search queries. This solves the problem by taking the complexity and research out of people's hands and allows our product to use people's budgets to fulfill their goals, rather than themselves.

Describe the architecture and design you have decided to use and why

Chosen architecture: **Layered Architecture**

- Maintainability and isolation
    - In a layered architecture, each layer is individual from each other, this means that changing one layer does not require a rewrite of the other systems.
    - Example: If we wanted to change AI models from gemini to claude, the frontend and database would remain the same.

- Parallel development
    - The frontend and backend can be built simultaneously, as long as developers agree on the interface between them they can work on each layer without conflicts.

- Layered architecture is easy to test
    - Since each layer is separate, we can run smaller unit tests on each layer without having to launch the entire software, for example we could test the budget calculator without needing to connect to the internet for the data layer or needing to launch the website.

- More secure
    - Generally, this type of software architecture can be more secure since the users never interact with the backend database. They interact with the UI, which talks to the API, which in turn talks to the database. The API can ensure that data sent to the database isn’t malicious or incorrect.

Describe the algorithms you will implement

Bayesian Average Algorithm

- $Score = \frac{(v \cdot R)+(m \cdot C)}{v+C}$

- R = average rating for the meal
- v = number of ratings for the meal
- C = minimum ratings threshold (e.g., 10)
- m = global average rating across all meals
- Explanation: We compute a Bayesian-averaged recommendation score for each meal to prevent meals with very few ratings from being unfairly over- or under-ranked. This score represents the global quality of a meal across all users and is updated whenever a new rating is submitted. It is stored directly on the meal document in Firestore and reused for all users.

User-Specific Diversity Penalty

- $\text{diversityWeight}_{u,m} = e^{-\lambda \cdot n}$
- $n_{u,m}$ = represents the number of times a user, *u,* has eaten a meal *m* within a fixed recent time window, not the length of the window itself.
- λ = diversity decay constant (0.5)
- $\mathbf{finalScore_{u,m} = recommendationScore_m \times diversityWeight_{u,m}}$
- Explanation: The final meal selection score combines a global Bayesian recommendation score with a user-specific diversity penalty to reduce repetition. Only meals consumed within a fixed recent time window are penalized, with each repeat exponentially down-weighting the meal's score to discourage overuse without banning it. This computation is performed dynamically in the backend at request time to generate varied, personalized meal plans.
- Our time window will be 14 days to discourage short-term repetition across weekly plans while allowing meals to resurface naturally over longer time horizons.

Describe the UI you will develop for the product

- There will be four primary pages that make up our website's UI: a landing page, a create account / log in page, a questionnaire, and a dashboard. The landing page will serve as a professional, eye-catching introduction to our project, including information about the project's purpose, how it works, and how the user can get started. If the user is interested in our project, they can create an account and be redirected to the questionnaire page, where they will answer questions regarding the food benefits they are receiving, the kind of food they're looking to make, and their health & fitness goals. If the user already has an account, they can log into it and immediately be redirected to their personalized dashboard, where they will find a weekly breakdown of their general monthly mealplan, including the ingredients they should purchase, the meals that they can make with said ingredients, and the monetary value of the meal plan that was generated for them.

How will you test your solution?

How will you prove, demonstrate, or argue that your solution solved the problem? In other words what kind of testing method will you use to provide assurance to your users?

- We would get users to use our product and review it to get an unbiased opinion.
    - We would ask friends or family members to use it for a week to see if the quality of meals was good, pricing was accurate, and overall user experience was positive.
- We can also use Playwright, which creates a fake user for end-to-end testing that automates each input and verifies the output is correct

What data will you use to test your application?

- We can create synthetic user profiles to test specific edge cases:
    - An extreme user who needs many calories but is on a low budget
    - A user with many dietary restrictions
    - An average user with a standard budget and no restrictions
- Real-world pricing from grocery stores
- Adversarial inputs:
    - Enter data such as impossibly high calorie counts to ensure the software handles it properly and does not crash.

What metrics will you consider to decide if your product is well-tested and of good quality?

- Tracking bug discovery during system testing, and a decrease in new bugs found over time will indicate higher quality software
- When generating a meal plan with hard constraints (budget, allergies, etc.), 100% of them should satisfy the constraints.
- 100% of the functional requirements should be met.

Project Sources & Citations

"Meal Kits Have Gotten Cheap, but Are They Cheaper than Groceries?" *CNET*, 2025, www.cnet.com/health/nutrition/meal-kits-have-gotten-cheap-but-are-they-cheaper-than-groceries/.

Campbell, Nina. "SNAP & Meal Kits: Are There Any Meal Kits That Accept EBT?" *AirTalk Wireless Blog*, July 2025, airtalkwireless.com/blog/are-there-any-meal-kits-that-accept-ebt. Accessed 15 Feb. 2026.

We used ChatGPT to help us write this document